

开发架构与框架技术概述—复习要点

1 课程概述与基础概念

1.1 课程基本信息

- 课程名称：软件开发架构平台（Software Development Architecture Platform），聚焦 Java EE 平台与企业级 Web 应用 [p.2]
- 前置知识（Prerequisites）：JavaSE、DB/SQL、JDBC、JSP/Servlet、HTML/CSS/XML、JavaScript、AJAX [p.2]
- 开发架构（Development Architecture，面向“人”）与系统架构（System Architecture，面向“机器”） [p.2]
 - 开发架构：系统分层 MVC、前后端分离、各种框架技术 [p.2]
 - 系统架构：数据缓存技术（Caching）、服务器集群部署（Clustering）、服务与 REST API 设计 [p.2]
- 考核方式：闭卷考试（60%）+ 实验（40%）；实验分组，每组 4–6 人 [p.4]
- 参考书籍：《Spring in Action》 [p.5]

2 框架技术的由来（Origins of Framework Technology）

2.1 Java Web MVC 架构

- 经典的 MVC（Model-View-Controller）开发架构已成为事实标准（de facto standard） [p.7]
- Model 层进一步细分为三部分 [p.8]:
 - 业务逻辑对象（Service / Business Bean）：完成业务逻辑 [p.8]
 - 数据持久化对象（DAO, Data Access Object）：实现数据持久化操作 [p.8]
 - 值对象（POJO, Plain Old Java Object）：仅用于表达数据的值对象 [p.8]
- 请求–响应链路：客户端请求 → Servlet → Service → DAO → POJO ↔ 数据库服务器；响应经 JSP 返回客户端 [p.8]
- 严格遵守 MVC 模式的 Web 项目应具有规范的文件/包结构 [p.9]

2.2 MVC 架构带来的两个核心问题

- 问题一：在开发过程中如何约束程序员遵循 MVC 架构 [p.10]
- 问题二：在使用 MVC 架构开发过程中如何简化和规范代码 [p.10]

2.3 Servlet 代码分析：相同点与不同点

- 相同点（不可变部分）：每个 Servlet 程序的流程基本一致，一般由三个部分组成 [p.13]:
 1. 从客户端获取数据 [p.13]
 2. 调用业务逻辑进行处理 [p.13]
 3. 根据处理的结果响应视图（JSP 页面） [p.13]
- 不同点（可变部分）：具体操作的数据不一样——获取数据的个数/类型、调用业务逻辑的方法、响应的 JSP 页面 [p.13]
- 表示层框架的设计思想：使用配置文件设定可变部分，将不可变部分用框架的方式确定下来，不由程序员编写 [p.13]
- 效果：约束了程序员遵循 MVC 架构规范，提高了 Web 应用程序表示层的开发效率，同时使项目具有较好的可维护性和可扩展性 [p.13]

3 三类框架及其职责

3.1 表示层框架（Presentation Layer Framework）

- 代表框架：Struts、Spring MVC [p.14]
- 解决目标：MVC 架构中 View 层（JSP 页面）和 Controller 层（Servlet 类）的规范与简化问题 [p.14]

3.2 持久层框架（Persistence Layer Framework）

- 代表框架：MyBatis、Hibernate、JPA [p.14]
- 解决目标：MVC 架构中 Model 层的数据访问对象（DAO 包）的规范与简化问题 [p.14]

3.3 容器类框架（Container Framework）

- 代表框架：Spring、EJB [p.14]
- 解决目标：MVC 各组件之间的耦合问题，包括横向耦合（同层组件间）和纵向耦合（跨层组件间） [p.14]

3.4 主流技术栈组合

- SSH：Spring + Struts2 + Hibernate [p.15]

- SSM（传统）：Spring + Spring MVC + MyBatis [p.15]
- SSM（现代/SpringBoot）：SpringBoot + Spring MVC + MyBatis [p.15]

4 Struts 框架的发展历史

- 2000 年 5 月，Craig R. McClanahan 向 Java 社区提交了一个 Web 框架，这是 Struts 1 的前身 [p.17]
- 2001 年 6 月，Struts 1.0 发布，成为 ASF Jakarta 项目的子项目 [p.17]
- 2004 年 3 月，Struts 升级为 ASF 中的顶级项目 [p.17]
- 2013 年 4 月，Struts 1.x 宣布 EOL（End of Life，生命周期结束） [p.17]
- 在此期间 WebWork 也在发展，随后拆分为 WebWork 2.0 和 XWork 1.0 两个框架 [p.17]
- 2006 年，出于多方原因考虑，WebWork 2.0 与 XWork 1.0 合并，并改名为 Struts 2 [p.17]
- 关键点：Struts 2 是基于 WebWork 2.3 进行改良的，本质上和 Struts 1.x 没有关系 [p.17]
- Struts 2 最新版本为 2.5.26，流行版本是 2.3.x [p.17]
- 目前表示层框架的实际应用中 Spring MVC 的使用率已经超过了 Struts 2 [p.17]

5 Struts 1 的基本原理

1. 导入 Struts 1.x 对应的依赖包（JAR 包） [p.19]
2. 在 web.xml 中配置 ActionServlet 接管请求 [p.20]
3. 创建 ActionForm 对象（用于封装表单数据） [p.21]
4. 创建 Action 对象（用于处理业务逻辑请求） [p.22]
5. 在 struts-config.xml 中完成配置（Action 映射、Forward 转发规则等） [p.23]

6 框架技术的侵入性（Framework Invasiveness）

- 定义（Definition）：在软件开发过程中，由于使用第三方框架技术而导致项目自身代码发生改变的程度，被称为框架/架构的侵入性 [p.24]
- 高侵入性（High Invasiveness） [p.24]:
 - 直接继承（Inheritance）或实现（Implementation）第三方框架的类或接口 [p.24]
 - 项目脱离框架时将无法运行 [p.24]
 - 导致项目重构（Refactoring）和单元测试（Unit Testing）效率降低，可维护性下降 [p.24]
 - 示例：Struts 1.x 等高侵入性框架 [p.24]
- 低侵入性（Low Invasiveness） [p.24]:
 - 通过反射（Reflection）、动态代理（Dynamic Proxy）等语言特性 [p.24]

- 结合 IoC（Inversion of Control，控制反转）和 AOP（Aspect-Oriented Programming，面向切面编程）等架构理论 [p.24]
- 动态调用第三方框架的类和接口，项目脱离框架时依然可以运行 [p.24]
- 提高可维护性和可扩展性，方便进行项目重构和单元测试等 [p.24]
- 示例：Spring、MyBatis 等低侵入性框架 [p.24]
- 软件架构设计理论中“高内聚、低耦合”（High Cohesion, Low Coupling）的主要目标也是为了降低侵入性 [p.24]

7 Struts 2 的基本原理

7.1 核心改进思想：约定优于配置（Convention over Configuration）

- 通过约定减少 XML 配置量，以简化开发过程 [p.25]

7.2 开发步骤

1. 导入 Struts 2.x 对应的依赖包 [p.26]
2. 在 web.xml 中配置 Filter 接管请求（关键变化：Struts 1 用 Servlet 接管，Struts 2 改用 Filter 接管） [p.27]
3. 创建业务领域对象（POJO 类，如 User），仅用于表达数据 [p.28]
4. 创建 Action 对象——与 Java Web 容器完全解耦（无需继承框架特定类，以 POJO 形式存在） [p.29]
5. 在 struts.xml 中完成配置 [p.30]

8 Spring MVC 框架基本原理

8.1 请求处理流程（七步）

1. 请求首先到达前端控制器（Front Controller / DispatcherServlet），委托给具体的控制器处理请求 [p.31]
2. 前端控制器通过查询处理器映射（Handler Mapping），找到 URL 对应的控制器 [p.31]
3. 控制器处理请求，包括处理数据、调用业务逻辑等 [p.31]
4. 控制器将模型数据（打包）和逻辑视图名（Logical View Name）返回给前端控制器 [p.31]
5. 视图解析器（View Resolver）将逻辑视图名匹配成具体的视图实现 [p.31]
6. 视图进行模型数据和视图实现的渲染（Rendering） [p.31]
7. 交付模型数据，给出 Web 响应 [p.31]

8.2 开发步骤与关键注解

1. 添加 Maven 依赖: spring-webmvc [p.32]
2. 配置 web.xml (注册 DispatcherServlet) [p.33]
3. 配置 applicationContext.xml (Spring Bean 上下文配置) [p.34]
4. 编写 Controller 类, 使用以下注解 [p.35]:
 - @Controller: 标注该类为一个 Servlet 控制器 [p.35]
 - @GetMapping: 注解 URL 映射, 如 `http://localhost:8080/hello` [p.35]
 - @ResponseBody: 注解方法返回值为直接以字符串内容进行响应 [p.35]

9 Struts 1 vs. Struts 2 vs. Spring MVC 对比

- **Struts 1:** 高侵入性, 需继承 Action/ActionForm 类; 使用 Servlet 接管请求; 在 struts-config.xml 中配置 [p.18–23]
- **Struts 2:** 低侵入性, Action 为 POJO; 使用 Filter 接管请求; 贯彻“约定优于配置”思想; 在 struts.xml 中配置 [p.25–30]
- **Spring MVC:** 低侵入性; 基于注解 (Annotation) 开发 (如 @Controller、@GetMapping、@ResponseBody); 以 DispatcherServlet 作为前端控制器; 在 applicationContext.xml 中配置 [p.31–35]

10 Maven

10.1 什么是 Maven

- Maven 是一种基于项目对象模型 (POM, Project Object Model) 的项目管理机制 [p.37]
- 通过简单的描述信息 (配置文件 pom.xml) 来管理项目的构建和模块间的依赖 [p.37]
- 核心功能一: 通过配置, 合理解决项目内部模块间和外部插件的依赖关系 (Dependency Management) [p.37]
- 核心功能二: 通过配置, 实现项目的自动化构建和部署运行 (Automated Build & Deploy) [p.37]

10.2 Maven 仓库 (Repository)

- 按所在地分为三类 [p.38]:
 - 本地仓库 (Local Repository) ——位于开发者本机 [p.38]
 - 中央仓库 (Central Repository) ——Maven 官方维护, 存放大部分开源项目依赖包 [p.38]
 - 远程仓库/私服 (Remote Repository / Private Server) ——组织或公司自行搭建的第三方仓库 [p.38]

- 中央仓库地址: <https://mvnrepository.com/> [p.38]
- Maven 通过 pom.xml 识别依赖, 将项目的相关依赖包下载到本地仓库中 [p.39]
- 如果项目所需要的依赖包中央仓库中没有, 可以在 pom.xml 中设置远程仓库 [p.39]
- 有些组织或公司为了依赖包的版本统一和缓解网络问题, 会构建自己的第三方仓库 (私服), 可在 settings.xml 配置文件中配置私服 [p.39]
- 依赖下载搜索顺序: 本地仓库 → 私服 → 中央仓库 → 远程仓库 [p.39]

10.3 POM 核心概念 (Project Object Model)

- 每个 Maven 项目都有一个唯一的 pom.xml 文件 [p.40]
- 每个 pom.xml 都有一个唯一的表示自身的坐标 (Coordinate), 由三部分组成 [p.40]:
 - groupId ——组织/项目组标识 [p.40]
 - artifactId ——项目/模块标识 [p.40]
 - version ——版本号 [p.40]
- pom.xml 文件大部分内容是描述项目的依赖 [p.40]:
 - 依赖通过 <dependencies> 子节点声明 [p.40]
 - 每个 <dependency> 表示一种依赖 [p.40]
 - 每个依赖也有其所依赖项目的坐标 (groupId、artifactId、version 三要素) 组成 [p.40]

10.4 Maven 约定目录结构 (Convention)

- target/ ——存放编译结果 (class 文件) [p.41]
- out/ ——存放输出结果 [p.41]
- src/ ——源代码目录, 分为项目自身源代码和测试源代码 [p.41]
- src/main/java ——项目自身 Java 源代码 [p.41]
- src/main/resources ——项目资源文件 [p.41]
- src/test/java ——测试用 Java 源代码 [p.41]
- src/test/resources ——测试用资源文件 [p.41]
- web/ ——Web 项目对应的 Web 目录 (HTML、CSS、JavaScript 等前端文件) [p.41]
- pom.xml ——位于项目根目录下 [p.41]

10.5 Maven 常用命令

- mvn compile ——编译 (Compile): 将 src/main/java 目录中的 Java 源代码编译成 class 到 target 目录下 [p.42]
- mvn test ——测试 (Test): 将 src/test/java 目录中的 Java 测试源代码编译成 class 到 target 目录下并运行 [p.42]
- mvn clean ——清理 (Clean): 删除 target 目录 [p.42]
- mvn package ——打包 (Package): 生成打包文件, 生成.jar 文件或.war 文件 [p.42]

- `mvn install` ——安装 (Install): 将打包文件上传到本地仓库 [p.42]
- `mvn deploy` ——部署/发布 (Deploy): 将打包文件上传到 Web 服务器或私服 [p.42]

10.6 Maven 构建与管理项目

- 构建 Maven 项目: 可以使用 Maven 新建项目, 也可以将已有项目转为 Maven 项目 [p.42]

11 项目自动化构建

- 主流自动化构建工具 (Automation Build Tools): Maven、Gradle、Ant 等 [p.43]

12 本章小结

- 开发架构与框架技术的发展 [p.44]
- 框架技术概览——以表示层框架为例: Struts 1 框架的基本原理; Struts 2 框架的基本原理; Spring MVC 框架的基本原理 [p.44]
- Maven 的基本原理和作用 [p.44]
- 其他自动化构建工具 [p.44]